Speech Diarization and Speaker Separation

A Multi-Approach Signal Processing Study

Rith Rajak — 24116085
Raj Shekhar Singh — 24116079

Abstract

This report looks at a Jupyter Notebook that tries to solve speech diarization, which is the problem of figuring out who spoke and when in an audio recording. We go through four different methods: splitting speakers based on pitch values, using a Wiener filter to separate overlapping speech, a supervised approach using Non-negative Matrix Factorization with harmonic templates, and a deep learning pipeline using the SepFormer model together with ECAPA-TDNN speaker embeddings. We also discuss where each method works well, where it breaks down, and why each one leads into the next.

1. Introduction

Speaker diarization is about answering a simple question: who spoke when? It is different from speech recognition, which tries to figure out what was said. Diarization just needs to label different speakers across time, without necessarily transcribing anything.

The notebook builds these pipelines from scratch, without using commercial APIs or black-box tools. This makes it easier to understand what each step is doing and why something might go wrong.

A common thread across the first three methods is the use of pitch, specifically the fundamental frequency (F0), which is the vibration rate of a person's vocal folds. Male voices typically have lower pitch than female voices, and this difference is what the simpler methods exploit. The fourth method, SepFormer, does not rely on pitch at all and instead uses patterns learned from large amounts of training data.

2. Signal Processing Basics

Speech changes constantly, the sounds shift on a timescale of around 10 to 30 milliseconds. To handle this, all four methods divide the audio into short overlapping windows called frames. The notebook uses a frame size of 2048 samples and a hop size of 512 samples. At a sample rate of 44100 Hz, this means each frame is about 46 ms long, and frames are computed every 11.6 ms with 75 percent overlap.

Pitch estimation uses pyin (probabilistic YIN). It works by looking for repeating patterns in the audio signal and estimating how often they repeat. It uses a Hidden Markov Model to make the pitch estimates smoother over time. For unvoiced sounds like "s" or "sh", pyin returns no value, so these frames need to be handled separately.

Voice Activity Detection (VAD) decides which parts of a recording contain actual speech and which are just silence. The notebook does this by computing the loudness (RMS

energy) of each frame and comparing it to a threshold set at 5 percent of the loudest moment in the recording.

3. Method 1: Pitch Threshold Diarization

The simplest method assumes that the two speakers take turns talking and do not overlap. It labels each voiced frame as either Speaker A or Speaker B based on pitch: frames below 180 Hz go to Speaker A, and frames above 180 Hz go to Speaker B.

This is a purely rule-based approach with no training needed. It works when the two speakers have clearly different pitch ranges. It fails when their pitch ranges overlap, or when one speaker raises their voice.

Unvoiced frames get labelled as unknown and are filled in using a two-pass technique. First, it scans left to right and assigns each unknown frame the label of the most recent known frame. Then it scans right to left to handle unknowns at the very beginning of the recording.

After gap filling, a median filter is applied to smooth out noisy frame-by-frame label switches. One documented bug in the notebook: the median filter outputs decimal numbers instead of whole numbers. If the result is not converted back to an integer before comparison, the comparison silently fails and produces wrong output without any error message.

code:

```
import librosa
import numpy as np
import soundfile as sf
from scipy.signal import medfilt

PITCH_SPLIT_HZ      = 180
SMOOTH_SECONDS      = 0.6
VAD_THRESHOLD_RATIO = 0.05
HOP_LENGTH          = 512
N_FFT               = 2048

def load_audio(path):
    y, sr = librosa.load(path, sr=None, mono=True)
    return y, sr

def extract_pitch(y, sr):
    f0, voiced_flag, _ = librosa.pyin(
        y, fmin=60, fmax=500, sr=sr, hop_length=HOP_LENGTH
    )
    return f0, voiced_flag

def compute_rms(y):
    rms = librosa.feature.rms(
        y=y, frame_length=N_FFT, hop_length=HOP_LENGTH
```

```
    )[0]
    return rms

def classify_frames(f0, voiced_flag, n_frames):
    labels = np.full(n_frames, -1, dtype=int)
    for i in range(n_frames):
        if voiced_flag[i] and not np.isnan(f0[i]):
            labels[i] = 0 if f0[i] < PITCH_SPLIT_HZ else 1
    return labels

def fill_unvoiced_gaps(labels):
    filled = labels.copy()
    last_known = -1
    for i in range(len(filled)):
        if filled[i] != -1:
            last_known = filled[i]
        elif last_known != -1:
            filled[i] = last_known
    last_known = -1
    for i in range(len(filled) - 1, -1, -1):
        if filled[i] != -1:
            last_known = filled[i]
        elif last_known != -1:
            filled[i] = last_known
    return filled

def smooth_labels(labels, sr):
    kernel = max(3, int((sr * SMOOTH_SECONDS) / HOP_LENGTH))
    if kernel % 2 == 0:
        kernel += 1
    # must cast back to int or comparisons silently fail
    smoothed = medfilt(
        labels.astype(float), kernel_size=kernel
    ).astype(int)
    return smoothed

def reconstruct_audio(y, labels, rms):
    vad_threshold = np.max(rms) * VAD_THRESHOLD_RATIO
    is_speech     = rms > vad_threshold
    n_frames      = len(labels)
    speaker_A     = np.zeros_like(y)
    speaker_B     = np.zeros_like(y)
    for i in range(n_frames):
        if not is_speech[i]:
            continue
        start = i * HOP_LENGTH
        end   = min(start + HOP_LENGTH, len(y))
        if labels[i] == 0:
```

```
            speaker_A[start:end] = y[start:end]
        else:
            speaker_B[start:end] = y[start:end]
    return speaker_A, speaker_B

def peak_normalise(audio, target_db=-1.0):
    peak = np.max(np.abs(audio))
    if peak == 0:
        return audio
    return audio * (10 ** (target_db / 20) / peak)

def separate_speakers(input_file, output_file_A, output_file_B):
    y, sr       = load_audio(input_file)
    f0, voiced  = extract_pitch(y, sr)
    rms         = compute_rms(y)
    n           = min(len(f0), len(rms))
    f0, voiced, rms = f0[:n], voiced[:n], rms[:n]
    labels      = classify_frames(f0, voiced, n)
    labels      = fill_unvoiced_gaps(labels)
    labels      = smooth_labels(labels, sr)
    spk_A, spk_B = reconstruct_audio(y, labels, rms)
    sf.write(output_file_A, peak_normalise(spk_A), sr)
    sf.write(output_file_B, peak_normalise(spk_B), sr)
```

4. Exploratory Data Analysis

Before running any separation, the notebook generates several diagnostic plots. Each one tests a specific assumption.

The waveform shows the overall shape of the audio over time. The FFT spectrum shows which frequencies have the most energy. The Mel spectrogram is a frequency versus time plot using a scale that matches human hearing. The Constant-Q Transform uses a log frequency scale which makes harmonic patterns easier to see visually. The F0 trajectory shows pitch over time, colour-coded by which speaker was assigned. The pitch histogram shows the distribution of pitch values and is the most important plot for validating the threshold. If there is a clear gap near 180 Hz between two peaks, the threshold is justified.

code:

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

HOP_LENGTH     = 512
N_FFT          = 2048
PITCH_SPLIT_HZ = 180

def plot_waveform(ax, y, sr):
```

```
    step = max(1, len(y) // 4000)
    t    = np.arange(0, len(y), step) / sr
    amp  = y[::step]
    ax.fill_between(t, amp, alpha=0.6, linewidth=0)
    ax.plot(t, amp, linewidth=0.4)
    ax.set_xlim(0, len(y) / sr)

def plot_fft(ax, y, sr, fft_max_hz=800):
    fft_mag   = np.abs(np.fft.rfft(y))
    fft_freqs = np.fft.rfftfreq(len(y), d=1.0 / sr)
    mask  = fft_freqs <= fft_max_hz
    freqs = fft_freqs[mask]
    mags  = fft_mag[mask] / fft_mag[mask].max()
    ax.plot(freqs, mags, linewidth=0.8)
    peak_idx = np.argsort(mags)[-3:][::-1]
    for idx in peak_idx:
        ax.annotate(
            f"{freqs[idx]:.0f} Hz",
            xy=(freqs[idx], mags[idx]),
            fontsize=7
        )

def plot_mel_spectrogram(ax, y, sr):
    mel    = librosa.feature.melspectrogram(
        y=y, sr=sr, n_mels=128,
        hop_length=HOP_LENGTH, n_fft=N_FFT
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    librosa.display.specshow(
        mel_db, sr=sr, hop_length=HOP_LENGTH,
        x_axis='time', y_axis='mel', ax=ax, cmap='magma'
    )

def plot_cqt(ax, y, sr, seg_s=10):
    seg  = y[: sr * seg_s]
    C    = np.abs(librosa.cqt(
        seg, sr=sr, hop_length=HOP_LENGTH,
        fmin=60, n_bins=84, bins_per_octave=24
    ))
    C_db = librosa.amplitude_to_db(C, ref=np.max)
    librosa.display.specshow(
        C_db, sr=sr, hop_length=HOP_LENGTH,
        x_axis='time', y_axis='cqt_hz',
        fmin=60, ax=ax, cmap='inferno'
    )
    ax.axhline(PITCH_SPLIT_HZ, linewidth=1.2, linestyle='--')

def plot_pitch_histogram(ax, y, sr):
```

```
    f0, voiced, _ = librosa.pyin(
        y, fmin=60, fmax=500, sr=sr, hop_length=HOP_LENGTH
    )
    f0_voiced = f0[~np.isnan(f0)]
    bins = np.arange(60, 420, 12)
    f0_A = f0_voiced[f0_voiced < PITCH_SPLIT_HZ]
    f0_B = f0_voiced[f0_voiced >= PITCH_SPLIT_HZ]
    ax.hist(f0_A, bins=bins, alpha=0.85, label='Speaker A')
    ax.hist(f0_B, bins=bins, alpha=0.85, label='Speaker B')
    ax.axvline(PITCH_SPLIT_HZ, linewidth=1.5,
               linestyle='--', label='Split 180 Hz')
    ax.legend()
```

5. Method 2: Wiener Harmonic Masking

When both speakers talk at the same time, each audio frame contains a mixture of both voices. Labelling a frame as belonging to one speaker is not enough; we need to separate the two voices within each frame.

This method takes advantage of the fact that voiced speech has a harmonic structure. When someone speaks at a certain pitch, the sound contains energy not just at that frequency but at all its multiples. Two speakers with different pitches will have their harmonics at different frequencies, which makes separation possible.

Instead of one pitch estimate per frame, this method estimates two: one in the range 60 to 160 Hz for Speaker A and one in 180 to 350 Hz for Speaker B. Using these estimates, the method builds a mask for each speaker and applies a Wiener filter to estimate each speaker's contribution.

There is a hard physical limit here. With the default settings, each frequency bin spans about 21.5 Hz. If two harmonics from different speakers fall within the same bin, no spectral mask can separate them. The notebook reduces the Gaussian width from 25 Hz to 8 Hz to reduce bleed between speakers.

code:

```
import numpy as np
from scipy import signal
from scipy.ndimage import uniform_filter1d, median_filter

N_FFT       = 2048
HOP         = 512
N_HARMONICS = 12
SIGMA_BASE  = 8.0
WIENER_EPS  = 1e-6
SMOOTH_T    = 5

def autocorr_f0_frame(frame, sr, fmin, fmax):
    frame = frame - frame.mean()
    if np.dot(frame, frame) < 1e-8:
```

```
        return np.nan, 0.0
    N   = len(frame)
    F   = np.fft.rfft(frame, n=2*N)
    ac  = np.fft.irfft(F * np.conj(F))[:N]
    ac /= (ac[0] + 1e-12)
    lag_min = int(np.ceil(sr / fmax))
    lag_max = min(int(np.floor(sr / fmin)), N - 1)
    if lag_min >= lag_max:
        return np.nan, 0.0
    seg      = ac[lag_min:lag_max + 1]
    peak_idx = np.argmax(seg)
    if seg[peak_idx] < 0.25:
        return np.nan, float(seg[peak_idx])
    lag = lag_min + peak_idx
    if 0 < peak_idx < len(seg) - 1:
        y1, y2, y3 = seg[peak_idx-1], seg[peak_idx], seg[peak_idx+1]
        denom = 2 * (2*y2 - y1 - y3)
        if denom != 0:
            lag += (y3 - y1) / denom
    return float(sr / lag), float(seg[peak_idx])

def build_harmonic_energy(f0, freq_bins, n_harmonics, sigma):
    n_freq, n_frames = len(freq_bins), len(f0)
    E = np.zeros((n_freq, n_frames), dtype=np.float32)
    for k in range(1, n_harmonics + 1):
        centres = k * f0[np.newaxis, :]
        diff    = freq_bins[:, np.newaxis] - centres
        E      += np.exp(-0.5 * (diff / sigma) ** 2).astype(np.float32)
    return E

def wiener_separate(f0_A, f0_B, freq_bins, stft_mix):
    n_frames = stft_mix.shape[1]
    f0_A = f0_A[:n_frames]
    f0_B = f0_B[:n_frames]
    E_A   = build_harmonic_energy(f0_A, freq_bins, N_HARMONICS, SIGMA_BASE)
    E_B   = build_harmonic_energy(f0_B, freq_bins, N_HARMONICS, SIGMA_BASE)
    denom = E_A + E_B + WIENER_EPS
    mask_A = uniform_filter1d(
        (E_A / denom).astype(np.float64), size=SMOOTH_T, axis=1
    ).astype(np.float32)
    mask_B = uniform_filter1d(
        (E_B / denom).astype(np.float64), size=SMOOTH_T, axis=1
    ).astype(np.float32)
    return stft_mix * mask_A, stft_mix * mask_B

freq_bins, _, Zxx = signal.stft(
    y, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP, window='hann'
)
```

```
out_A_stft, out_B_stft = wiener_separate(f0_A, f0_B, freq_bins, Zxx)
_, audio_A = signal.istft(
    out_A_stft, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP
)
_, audio_B = signal.istft(
    out_B_stft, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP
)
```

6. Method 3: Supervised NMF with Harmonic Dictionary

To get around the bin-width limit, the NMF method doubles the FFT size to 4096, which cuts the bin width in half to about 10.8 Hz. Two harmonics that were previously in the same bin are now in adjacent, separable bins.

Rather than learning patterns from data, this method uses a hand-crafted dictionary of harmonic templates. Each column in the dictionary represents what a speaker voicing at a specific pitch would look like in the spectrogram. Speaker A templates span 65 to 160 Hz and Speaker B spans 180 to 350 Hz, giving 67 templates total.

NMF tries to express the spectrogram as a combination of these templates with non-negative weights. The notebook uses a KL-divergence version of NMF, which handles large outlier values more gracefully and produces sparser activations. The algorithm runs for 120 iterations and prints the loss every 30 steps so convergence can be checked.

code:

```
import numpy as np
from scipy import signal
from scipy.ndimage import uniform_filter1d
import time

N_FFT     = 4096
HOP       = 512
N_HARM    = 14
SIGMA_NMF = 5.0
NMF_ITER  = 120
SMOOTH_T  = 5
F0_GRID_A = np.arange(65, 161, 3, dtype=np.float32)
F0_GRID_B = np.arange(180, 351, 5, dtype=np.float32)

def make_harmonic_dict(f0_grid, freq_bins, n_harm=14, sigma=5.0):
    W = np.zeros((len(freq_bins), len(f0_grid)), dtype=np.float64)
    for j, f0 in enumerate(f0_grid):
        for k in range(1, n_harm + 1):
            center = k * f0
            if center > freq_bins[-1]:
                break
            W[:, j] += np.exp(
```

```
                -0.5 * ((freq_bins - center) / sigma) ** 2
            )
    col_sum = W.sum(axis=0, keepdims=True)
    W /= np.where(col_sum > 1e-12, col_sum, 1.0)
    return W

def nmf_supervised_kl(V, W, n_iter=120):
    n_comp    = W.shape[1]
    H         = (np.random.rand(n_comp, V.shape[1]).astype(np.float64)
                 * (V.mean() / n_comp + 0.01))
    W_col_sum = W.sum(axis=0)[:, np.newaxis]
    t0        = time.time()
    for it in range(1, n_iter + 1):
        WH = np.maximum(W @ H, 1e-10)
        H *= (W.T @ (V / WH)) / (W_col_sum + 1e-10)
        H  = np.maximum(H, 1e-10)
        if it % 30 == 0:
            kl = np.mean(V * np.log(V / WH + 1e-10) - V + WH)
            print(f"iter {it}/{n_iter}  KL={kl:.4e}  "
                  f"t={time.time()-t0:.1f}s")
    return H.astype(np.float32)

freq_bins, _, Zxx = signal.stft(
    y, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP, window='hann'
)
V   = np.abs(Zxx).astype(np.float64) + 1e-10
W_A = make_harmonic_dict(F0_GRID_A, freq_bins, N_HARM, SIGMA_NMF)
W_B = make_harmonic_dict(F0_GRID_B, freq_bins, N_HARM, SIGMA_NMF)
R_A = W_A.shape[1]
W   = np.hstack([W_A, W_B])

H   = nmf_supervised_kl(V, W, NMF_ITER)
V_A = (W_A @ H[:R_A].astype(np.float64)).astype(np.float32)
V_B = (W_B @ H[R_A:].astype(np.float64)).astype(np.float32)

mask_A = uniform_filter1d(
    (V_A / (V_A + V_B + 1e-8)).astype(np.float64), SMOOTH_T, 1
).astype(np.float32)
mask_B = uniform_filter1d(
    (V_B / (V_A + V_B + 1e-8)).astype(np.float64), SMOOTH_T, 1
).astype(np.float32)

_, aA = signal.istft(
    Zxx * mask_A, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP
)
_, aB = signal.istft(
    Zxx * mask_B, fs=sr, nperseg=N_FFT, noverlap=N_FFT - HOP
)
```

7. Method 4: SepFormer and ECAPA-TDNN

SepFormer is a deep learning model specifically designed for speech separation. It was trained on a large dataset of mixed speech recordings and learns to separate voices without any assumptions about pitch or harmonic structure. It uses a transformer architecture with self-attention to capture both short and long-range patterns in the audio.

The main risk is domain mismatch. The model was trained on clean studio recordings, so it may not work well on real-world audio with background noise or echoes.

SepFormer produces two separated audio tracks but does not know which one is Speaker A and which is Speaker B. To solve this, the notebook uses ECAPA-TDNN, a speaker verification model that converts an audio clip into a vector representing that speaker's voice characteristics. Embeddings are computed for both SepFormer outputs and compared to rough reference tracks from the NMF stage. The assignment that maximises overall similarity is chosen.

After separation, a spectral subtraction pass is applied to reduce any remaining bleed between channels.

code:

```
import torch
import torchaudio
import torchaudio.functional as F_audio
import numpy as np
from speechbrain.inference.separation import SepformerSeparation
from speechbrain.inference.speaker import EncoderClassifier

device = "cuda" if torch.cuda.is_available() else "cpu"

def load_audio(path, target_sr=8000):
    wav, sr = torchaudio.load(path)
    if wav.shape[0] > 1:
        wav = wav.mean(dim=0, keepdim=True)
    if sr != target_sr:
        wav = F_audio.resample(wav, sr, target_sr)
    return wav, target_sr

sep_model = SepformerSeparation.from_hparams(
    source="speechbrain/sepformer-wsj02mix",
    savedir="pretrained/sepformer-wsj02mix",
    run_opts={"device": device}
)

mix, sr = load_audio(MIX_PATH, 8000)
torchaudio.save("tmp_mix_8k.wav", mix, 8000)
est   = sep_model.separate_file(path="tmp_mix_8k.wav")
src1  = est[:, :, 0]
src2  = est[:, :, 1]
```

```
spk_encoder = EncoderClassifier.from_hparams(
    source="speechbrain/spkrec-ecapa-voxceleb",
    savedir="pretrained/spkrec-ecapa",
    run_opts={"device": device}
)

def get_embedding(wav_tensor):
    emb = spk_encoder.encode_batch(
        wav_tensor.to(device)
    ).squeeze()
    return emb / (emb.norm() + 1e-12)

def cosine_sim(a, b):
    return float((a * b).sum())

emb_ref_a = get_embedding(ref_a)
emb_ref_b = get_embedding(ref_b)
emb_src1  = get_embedding(src1)
emb_src2  = get_embedding(src2)

if (cosine_sim(emb_src1, emb_ref_a)
        + cosine_sim(emb_src2, emb_ref_b)
        >= cosine_sim(emb_src1, emb_ref_b)
        + cosine_sim(emb_src2, emb_ref_a)):
    final_A, final_B = src1, src2
else:
    final_A, final_B = src2, src1

def spectral_subtract(sig, noise, alpha=1.0, beta=0.003):
    sig_np = sig.squeeze().cpu().numpy()
    noi_np = noise.squeeze().cpu().numpy()
    N, hop = 2048, 512
    win    = np.hanning(N)

    def stft(x):
        return np.array([
            np.fft.rfft(x[i:i+N] * win)
            for i in range(0, len(x) - N, hop)
        ]).T

    def istft(S, length):
        frames = np.fft.irfft(S.T, n=N) * win
        out    = np.zeros(length + N)
        for i, f in enumerate(frames):
            out[i*hop:i*hop+N] += f
        return out[:length]

    S_sig     = stft(sig_np)
```

```
    S_noi     = stft(noi_np)
    noise_est = np.median(
        np.abs(S_noi), axis=1, keepdims=True
    ) * alpha
    mag_clean = np.maximum(
        np.abs(S_sig) - noise_est, beta * np.abs(S_sig)
    )
    S_clean = mag_clean * np.exp(1j * np.angle(S_sig))
    out     = istft(S_clean, len(sig_np))
    return torch.tensor(out, dtype=torch.float32).unsqueeze(0)

final_A_clean = spectral_subtract(final_A, final_B)
final_B_clean = spectral_subtract(final_B, final_A)
```

8. Post-Processing

After separation, the notebook applies several improvements to the output audio.

Automatic Gain Control smooths out volume differences over time using a one-pole recursive envelope follower with separate attack and release time constants. It reacts quickly to sudden loudness increases but fades slowly, matching how human ears perceive volume changes.

Pitch shifting uses a phase vocoder to shift pitch without changing speed. The signal is time-stretched by the desired pitch ratio and then resampled back to the original length. Phase coherence is maintained by accumulating instantaneous frequency estimates.

VAD gating cuts out very quiet sections using a fixed loudness threshold, with a 150 ms hold time to prevent cutting out mid-word.

Spectral subtraction denoising estimates noise from the quietest frames and subtracts it from the rest.

Speech equalisation applies a high-pass filter to remove low-frequency rumble, a boost around 1 to 3 kHz where consonants are clearest, and an optional high-frequency cut.

code:

```
import numpy as np
import soundfile as sf
import librosa
from scipy.signal import butter, sosfilt, medfilt

y, sr = librosa.load(INPUT_WAV, sr=None, mono=True)

# (a) Automatic Gain Control
def envelope_follower(signal, sr, attack_ms=10, release_ms=100):
    attack_coef  = np.exp(-1.0 / (sr * attack_ms  / 1000.0))
    release_coef = np.exp(-1.0 / (sr * release_ms / 1000.0))
    env  = np.zeros_like(signal)
    prev = 0.0
```

```
    rect = np.abs(signal)
    for i in range(len(rect)):
        if rect[i] > prev:
            env[i] = ((1 - attack_coef) * rect[i]
                      + attack_coef * prev)
        else:
            env[i] = ((1 - release_coef) * rect[i]
                      + release_coef * prev)
        prev = env[i]
    return env

def agc(signal, sr, target_rms=0.1,
        attack_ms=10, release_ms=150):
    env  = np.maximum(
        envelope_follower(signal, sr, attack_ms, release_ms), 1e-6
    )
    gain = np.minimum(target_rms / env, 10.0)
    return np.clip(signal * gain, -1.0, 1.0)

y_agc = agc(y, sr)
sf.write("a_agc_output.wav", y_agc, sr)

# (b) Pitch shift via phase vocoder
def pitch_shift_vocoder(signal, sr, semitones,
                        n_fft=2048, hop=512):
    r         = 2.0 ** (semitones / 12.0)
    win       = np.hanning(n_fft).astype(np.float64)
    mag       = n_fft // 2 + 1
    freqs     = 2 * np.pi * np.arange(mag) / n_fft
    hop_syn   = int(hop * r)
    n_frames  = 1 + (len(signal) - n_fft) // hop
    out_len   = n_fft + (n_frames - 1) * hop_syn
    output    = np.zeros(out_len, dtype=np.float64)
    norm      = np.zeros(out_len, dtype=np.float64)
    phase     = np.zeros(mag)
    phase_acc = np.zeros(mag)
    for i in range(n_frames):
        frame      = (signal[i*hop:i*hop+n_fft].astype(np.float64)
                      * win)
        X          = np.fft.rfft(frame)
        mag_X      = np.abs(X)
        phs_X      = np.angle(X)
        delta      = phs_X - phase - freqs * hop
        delta     -= 2*np.pi * np.round(delta / (2*np.pi))
        phase_acc += freqs * hop_syn + delta * (hop_syn / hop)
        phase      = phs_X
        Y          = mag_X * np.exp(1j * phase_acc)
        frame_out  = np.fft.irfft(Y) * win
```

```
        pos        = i * hop_syn
        output[pos:pos+n_fft] += frame_out
        norm[pos:pos+n_fft]   += win**2
    output /= np.maximum(norm, 1e-8)
    output   = output[:out_len].astype(np.float32)
    indices  = np.linspace(0, len(output)-1, len(signal))
    return np.interp(indices, np.arange(len(output)), output)

sf.write("b_pitch_up2.wav",
         pitch_shift_vocoder(y, sr, semitones=+2), sr)
sf.write("b_pitch_down3.wav",
         pitch_shift_vocoder(y, sr, semitones=-3), sr)

# (d) VAD gate
def vad_gate(signal, sr, threshold_db=-40,
             frame_ms=20, hold_ms=200):
    frame_len   = int(sr * frame_ms / 1000)
    hold_frames = int(hold_ms / frame_ms)
    n_frames    = len(signal) // frame_len
    gate        = np.zeros(n_frames, dtype=bool)
    for i in range(n_frames):
        rms = np.sqrt(np.mean(
            signal[i*frame_len:(i+1)*frame_len]**2
        ) + 1e-12)
        gate[i] = (20 * np.log10(rms) >= threshold_db)
    held = gate.copy()
    for i in range(n_frames):
        if gate[i]:
            held[i:i+hold_frames] = True
    output = signal.copy()
    for i in range(n_frames):
        if not held[i]:
            output[i*frame_len:(i+1)*frame_len] = 0.0
    return output

sf.write("d_vad_gated.wav",
         vad_gate(y, sr, threshold_db=-42), sr)

# (e) F0 via autocorrelation
def autocorr_f0(frame, sr, f_min=60, f_max=500):
    frame = frame - frame.mean()
    if frame.std() < 1e-6:
        return np.nan, 0.0
    N  = len(frame)
    F  = np.fft.rfft(frame, n=2*N)
    ac = np.fft.irfft(F * np.conj(F))[:N]
    ac /= (ac[0] + 1e-12)
    lo = int(np.ceil(sr / f_max))
```

```
    hi = min(int(np.floor(sr / f_min)), N - 2)
    if lo >= hi:
        return np.nan, 0.0
    seg = ac[lo:hi+1]
    pi  = np.argmax(seg)
    if seg[pi] < 0.3:
        return np.nan, float(seg[pi])
    lag = lo + pi
    if 0 < pi < len(seg) - 1:
        y1, y2, y3 = seg[pi-1], seg[pi], seg[pi+1]
        d = 2 * (2*y2 - y1 - y3)
        if d != 0:
            lag += (y3 - y1) / d
    return float(sr / lag), float(seg[pi])

# (f) Noise reduction and speech EQ
def spectral_subtraction_denoise(signal, sr, alpha=1.5,
                                 beta=0.002, n_quiet=20):
    N, hop = 2048, 512
    win    = np.hanning(N).astype(np.float64)
    frames, phases = [], []
    for i in range(0, len(signal) - N, hop):
        spec = np.fft.rfft(
            signal[i:i+N].astype(np.float64) * win
        )
        frames.append(np.abs(spec))
        phases.append(np.angle(spec))
    frames    = np.array(frames)
    phases    = np.array(phases)
    noise_prof = (frames[
        np.argsort(frames.mean(axis=1))[:n_quiet]
    ].mean(axis=0) * alpha)
    mag_clean = np.maximum(
        frames - noise_prof[None, :], beta * frames
    )
    out = np.zeros(len(signal) + N, dtype=np.float64)
    nrm = np.zeros_like(out)
    for i, (mag, ph) in enumerate(zip(mag_clean, phases)):
        spec = mag * np.exp(1j * ph)
        frm  = np.fft.irfft(spec, n=N) * win
        out[i*hop:i*hop+N] += frm
        nrm[i*hop:i*hop+N] += win**2
    return (out / np.maximum(nrm, 1e-8))[:len(signal)].astype(
        np.float32
    )

def speech_eq(signal, sr):
    sos_hp = butter(4, 80, btype='high', fs=sr, output='sos')
```

```
    out    = sosfilt(sos_hp, signal.astype(np.float64))
    sos_bp = butter(
        2, [1000, 3000], btype='band', fs=sr, output='sos'
    )
    out    = out + 0.5 * sosfilt(sos_bp, out)
    if sr > 16000:
        sos_lp = butter(
            4, 8000, btype='low', fs=sr, output='sos'
        )
        out    = sosfilt(sos_lp, out)
    return out.astype(np.float32)

y_denoised = spectral_subtraction_denoise(y, sr)
y_eq       = speech_eq(y_denoised, sr)
peak       = np.abs(y_eq).max()
if peak > 0:
    y_eq = y_eq * (10**(-1/20) / peak)
sf.write("f_denoised_eq.wav", np.clip(y_eq, -1, 1), sr)
```

9. Comparison and Limitations

Method	Core Idea	Main Assumption	Where it Fails
Pitch Threshold	pyin + hard split	Speakers have different pitches	Similar pitch ranges
Wiener Masking	Harmonic Gaussians	Harmonics in different bins	Bins too wide (21.5 Hz)
Supervised NMF	KL-NMF, dictionary	Sparse harmonic decomposition	Fixed-grid dictionary
SepFormer	Transformer model	Training data matches input	Domain mismatch

All four methods share some common limitations. They all assume exactly two speakers. There is no ground truth available to measure how well the diarization actually worked. All methods process the full recording at once and cannot run in real time. Noise estimation assumes that noise is constant and that some parts of the recording are completely silent, which is often not the case.

10. Conclusion

This notebook builds a complete speech diarization system across four progressively more sophisticated methods. What makes it useful is the way each method's failure directly motivates the next step.

The pitch threshold method is clean and easy to understand but fails when voices are similar. Wiener masking handles simultaneous speech but runs into a physical frequency resolution limit. NMF gets around this by increasing the resolution. SepFormer sidesteps

all these issues by learning directly from data, but introduces its own risk of poor generalisation.

The most memorable implementation lesson is the median filter type-cast bug: wrong audio output, no error, no warning. It is a good reminder that in signal processing, correctness means understanding the numerical behaviour of every function used, not just getting the code to run.